# Lesson Plan: Module 2

## FPGA RTL Design, Signal Ingestion Pipeline, and Simulation Verification

---

**An Educational Framework for AI-Driven Signal Processing**


| | |
|---|---|
| **Student:** | Jude Eschete |
| **Project Advisor:** | Dr. Bernard Yett |
| **Program:** | M.S. Computer Engineering |
| **Institution:** | Stevens Institute of Technology |
| **Date Range:** | March 15 – March 28, 2026 |



This document presents the lesson plan for the second module of the capstone project, covering the design and simulation of the FPGA-side signal processing pipeline on the Digilent Nexys A7-100T. Students implement UART-based data ingestion, packet parsing with hardware CRC verification, and a feature extraction datapath in VHDL, then verify the complete pipeline through simulation testbenches driven by recorded beacon data from Module 1.



JEschete@stevens.edu

# Contents


**1 Module Overview** **1**
1.1 Context Within the Project . . . 1
1.2 Prerequisites . . . 1

**2 Weekly Schedule** **3**

**3 Week 1: UART Receiver and Clock Domain Management** **4**
3.1 Objectives . . . 4
3.2 The Clock Domain Crossing Problem . . . 4
3.2.1 Dual-Flip-Flop Synchronizer . . . 4
3.3 UART Receiver Architecture . . . 5
3.3.1 Receiver State Machine . . . 5
3.3.2 Key Design Parameters . . . 5
3.3.3 Entity Interface . . . 5
3.4 Receive FIFO . . . 6
3.5 Week 1 Integration . . . 7

**4 Week 2: Packet Parser and Frame Integrity Verification** **9**
4.1 Objectives . . . 9
4.2 Frame Structure Recap . . . 9
4.3 Packet Parser State Machine . . . 9
4.3.1 Parser Entity Interface . . . 9
4.4 Hardware CRC Implementation . . . 9
4.4.1 CRC-8 (Frame Transport Integrity) . . . 10
4.4.2 CRC-16 (Beacon Payload Integrity) . . . 11
4.4.3 CRC Verification Timing . . . 12
4.5 Integrated Parser Simulation . . . 13

**5 Week 3: Feature Extraction Datapath** **14**
5.1 Objectives . . . 14
5.2 Feature Vector Design . . . 14
5.2.1 Feature Vector Format . . . 14
5.2.2 Computed Features . . . 14
5.3 Per-Beacon State Registers . . . 16
5.4 Feature Storage: Dual-Port Block RAM . . . 16
5.4.1 Write Logic . . . 17
5.4.2 Read / Export Interface . . . 17
5.5 Beacon Prioritization Logic . . . 17
5.6 Week 3 Pipeline Architecture . . . 18

**6 Week 4: Simulation, Verification, and Resource Optimization** **19**
6.1 Objectives . . . 19
6.2 End-to-End Testbench Architecture . . . 19
6.2.1 Stimulus Generation . . . 19
6.2.2 Output Checking . . . 19

6.3 Verification Scenarios . . . 19
6.4 Resource Utilization Analysis . . . 19
6.4.1 Expected Resource Budget . . . 20
6.5 Timing Analysis . . . 20
6.6 Design Documentation Requirements . . . 22

**7 Hardware and Software Reference 23**
7.1 FPGA Pin Assignments . . . 23

**8 Assessment Rubric 23**

**9 Recommended Resources 24**
9.1 Datasheets and Technical References . . . 24
9.2 VHDL Design References . . . 25
9.3 CRC Implementation References . . . 25
9.4 Application Notes . . . 25

**10 Transition to Module 3 26**

# 1 Module Overview

**Module 2 Learning Objectives**

Upon completion of this module, the student (or future learner following this curriculum) will be able to:

**LO-1** Implement a UART receiver module in VHDL that reliably captures asynchronous serial data at 115200 baud and interfaces with the FPGA's 100 MHz system clock domain through proper synchronization techniques.

**LO-2** Design and implement a packet parser state machine that detects frame delimiters, extracts fixed-length fields, and routes parsed data to downstream processing stages.

**LO-3** Implement hardware CRC verification (CRC-8 for the receiver frame, CRC-16 for the beacon payload) and use the results to gate downstream processing, rejecting corrupted frames in real time.

**LO-4** Architect a feature extraction datapath that transforms parsed packet fields into a structured feature vector suitable for the machine learning classification pipeline in Module 4.

**LO-5** Develop simulation testbenches in VHDL that use recorded beacon data from Module 1 as stimulus, verify functional correctness of each pipeline stage, and identify timing or resource issues before hardware deployment.

**LO-6** Analyze FPGA resource utilization (LUTs, flip-flops, block RAM, DSP slices) and timing reports to make informed design trade-offs within the Artix-7 fabric constraints.

## 1.1 Context Within the Project

Module 2 builds directly on the hardware platform validated in Module 1. The UART interface between the receiver STM32 and the Nexys A7 FPGA, characterized in Module 1 Week 4, becomes the input to the RTL processing pipeline developed here. The beacon packet format (32 bytes, Table 4 in Module 1) and the receiver-to-FPGA forwarding frame (43 bytes, Table 6 in Module 1) define the exact data structures that the FPGA logic must parse.

This module represents the first point in the curriculum where students work entirely within the FPGA design domain, but the design decisions they make are constrained by upstream choices from Module 1 (baud rate, packet format, interface protocol) and must anticipate downstream requirements in Module 3 (system integration throughput) and Module 4 (feature vector format for ML training). This cross-module dependency chain—firmware choices constraining RTL design, which in turn shapes the ML data pipeline—is the central pedagogical theme of the framework.

## 1.2 Prerequisites

Students beginning this module should have:

- Completed Module 1 with a working beacon–receiver subsystem and validated FPGA toolchain (Vivado, Nexys A7 bitstream generation confirmed).

- Proficiency in VHDL (entity/architecture declarations, signal assignments, process blocks, component instantiation) or equivalent Verilog fluency.
- Understanding of synchronous digital design principles: clock domains, setup/hold timing, finite state machines, and registered outputs.
- Familiarity with simulation concepts (testbench structure, stimulus generation, waveform analysis) in Vivado Simulator or ModelSim.
- Access to the hardware and software listed in Section 7.
- Recorded beacon data files (UART byte captures) from Module 1 Week 4 validation exercises.

## 2 Weekly Schedule

The module spans two weeks (four working sessions) of intensive RTL development, with each session progressing from lower-level interface logic to higher-level data processing. Simulation verification is integrated throughout rather than deferred to the end.

Table 1: Module 2 weekly breakdown.

| Week | Focus Area | Key Activities |
|---|---|---|
| 1 | UART Receiver & Clock Domain Management | Implement UART RX module with oversampling; design dual-flip-flop synchronizer for clock domain crossing; implement byte-level receive FIFO; simulate and verify against recorded Module 1 byte streams. |
| 2 | Packet Parser & CRC Verification | Implement frame delimiter detection and packet parser state machine; design CRC-8 and CRC-16 hardware verification modules; integrate parser with UART receiver; simulate end-to-end frame reception with both valid and corrupted inputs. |
| 3 | Feature Extraction Datapath | Design feature vector format for ML pipeline; implement field extraction and feature register file; add dual-port BRAM feature storage for multi-beacon accumulation; implement beacon prioritization logic based on signal quality metrics. |
| 4 | Simulation, Verification & Optimization | Build comprehensive testbenches using full Module 1 data captures; run end-to-end pipeline simulation; analyze resource utilization and timing reports; optimize critical paths; document design and prepare RTL for Module 3 integration. |

# 3 Week 1: UART Receiver and Clock Domain Management

## 3.1 Objectives

- Implement a UART receiver module in VHDL capable of reliable 115200-baud reception on the Nexys A7's 100 MHz clock domain.
- Design and implement a dual-flip-flop synchronizer for the asynchronous UART input.
- Implement a small receive FIFO to decouple byte reception timing from downstream consumption.
- Verify the UART receiver in simulation against recorded byte sequences from Module 1.

## 3.2 The Clock Domain Crossing Problem

The receiver-to-FPGA interface validated in Module 1 uses UART at 115200 baud. From the FPGA's perspective, the incoming serial line is an asynchronous signal: it transitions according to the STM32's clock, which bears no phase or frequency relationship to the Nexys A7's 100 MHz oscillator. Sampling this signal directly with a 100 MHz register risks metastability—a condition where the register output is neither a clean logic 0 nor logic 1 for an indeterminate settling period.

**Why This Matters Beyond UART**

Clock domain crossing is not merely a UART implementation detail. It is the foundational challenge of any system that ingests data from an external source—precisely the situation in every RF receiver, sensor interface, and multi-chip pipeline. Students who understand the synchronization problem here will recognize it again when connecting multiple FPGA subsystems in Module 3 and when evaluating real-time inference latency in Module 5. The UART interface is intentionally chosen as the simplest context in which to encounter this universal FPGA design challenge.

### 3.2.1 Dual-Flip-Flop Synchronizer

The standard mitigation for single-bit asynchronous inputs is a dual-flip-flop (two-stage) synchronizer. The first flip-flop may go metastable, but the second flip-flop samples the (now quasi-stable) output of the first, reducing the probability of metastable propagation to negligible levels for any practical clock frequency.

$$\text{MTBF} = \frac{e^{t_r/\tau}}{T_0 \cdot f_{\text{clk}} \cdot f_{\text{data}}} \tag{1}$$

where $t_r$ is the resolution time (one clock period for the second stage), $\tau$ is the metastability time constant of the flip-flop (technology-dependent, typically 20–40 ps for Artix-7), $T_0$ is the metastability window, $f_{\text{clk}}$ is the sampling clock frequency, and $f_{\text{data}}$ is the data transition rate. For the Artix-7 at 100 MHz sampling a 115200-baud signal, the MTBF of a two-stage synchronizer exceeds $10^{15}$ years—effectively zero risk.

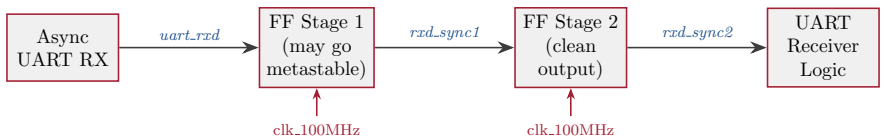


Figure 1: Dual-flip-flop synchronizer for the asynchronous UART input. Both stages are clocked by the FPGA's 100 MHz system clock.

**Common Pitfalls**

- **Skipping the synchronizer:** In simulation, the asynchronous input appears perfectly clean because the simulator resolves signals in zero time. The design will *appear* to work without a synchronizer in simulation but will exhibit intermittent failures on hardware. Always include the synchronizer, even though simulation cannot demonstrate the need for it.
- **Synthesis attributes:** Mark the synchronizer flip-flops with the `ASYNC_REG` attribute so that Vivado places them in the same slice, minimizing routing delay between stages:

```
attribute ASYNC_REG : string;
attribute ASYNC_REG of rxd_sync1 :  signal is "TRUE";
attribute ASYNC_REG of rxd_sync2 :  signal is "TRUE";
```

- **Do not synchronize multi-bit buses this way.** The dual-FF synchronizer is valid only for single-bit signals. Multi-bit data crossing clock domains requires a FIFO with gray-coded pointers or a handshake protocol.

## 3.3 UART Receiver Architecture

The UART receiver samples the synchronized serial input at 16× the baud rate (oversampling) to reliably detect the start bit edge and center-sample each data bit. At 115200 baud, the bit period is:

$$T_{\text{bit}} = \frac{1}{115200} \approx 8.68\ \mu\text{s} \tag{2}$$

With a 100 MHz system clock, one bit period spans:

$$N_{\text{clk/bit}} = \frac{100 \times 10^6}{115200} \approx 868 \text{ clock cycles} \tag{3}$$

The 16× oversampling divider is:

$$N_{\text{oversample}} = \frac{N_{\text{clk/bit}}}{16} = \frac{868}{16} \approx 54 \text{ clock cycles per sample} \tag{4}$$

### 3.3.1 Receiver State Machine

### 3.3.2 Key Design Parameters

### 3.3.3 Entity Interface

The UART receiver module exposes the following port interface:

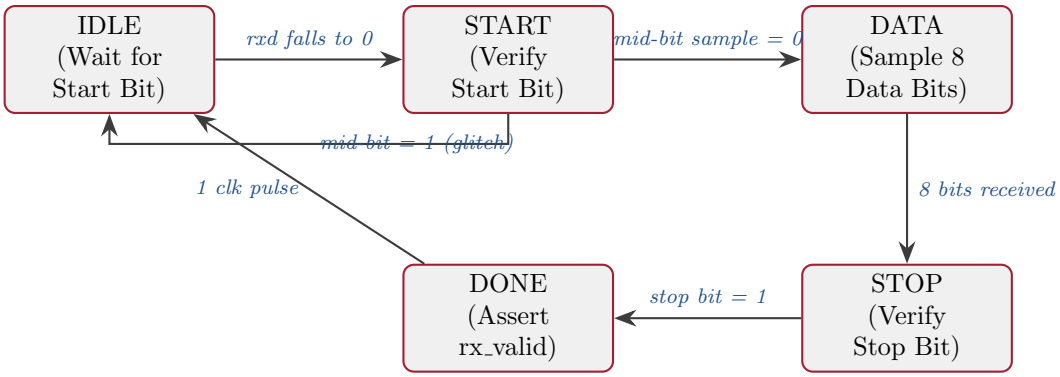


Figure 2: UART receiver state machine. The receiver waits for a falling edge (start bit), verifies at mid-bit, samples 8 data bits at their centers, checks the stop bit, and asserts a one-cycle `rx_valid` pulse with the received byte on `rx_data[7:0]`.

Table 2: UART receiver design parameters.

| **Parameter** | **Value** | **Rationale** |
|---|---|---|
| System Clock | 100 MHz | Nexys A7 on-board oscillator |
| Baud Rate | 115200 | Matched to Module 1 receiver firmware |
| Data Bits | 8 | Standard byte-oriented transfer |
| Parity | None | CRC provides integrity checking at frame level |
| Stop Bits | 1 | Standard configuration |
| Oversampling | 16× | Provides ±3% baud rate tolerance |
| Clocks per Sample | 54 | $\lfloor 100\text{M}/(115200 \times 16) \rfloor$ |

Table 3: UART receiver entity ports.

| **Port** | **Direction** | **Width** | **Description** |
|---|---|---|---|
| `clk` | in | 1 | 100 MHz system clock |
| `rst` | in | 1 | Synchronous active-high reset |
| `uart_rxd` | in | 1 | Raw UART serial input (asynchronous) |
| `rx_data` | out | 8 | Received byte (valid when `rx_valid` = '1') |
| `rx_valid` | out | 1 | One-cycle pulse indicating a valid byte is available |
| `rx_error` | out | 1 | Framing error (stop bit not detected) |

### 3.4 Receive FIFO

A small synchronous FIFO decouples the byte-by-byte arrival rate of the UART from the downstream packet parser, which consumes bytes in bursts (one complete frame at a time). At 115200 baud, one byte arrives every ∼86.8 $\mu$s (10 bit times including start and stop). A 43-byte receiver frame takes approximately 3.73 ms to arrive. The FIFO must buffer at least one complete frame.

Table 4: Receive FIFO design parameters.

| Parameter | Value | Rationale |
|---|---|---|
| Width | 8 bits | Byte-oriented data |
| Depth | 64 entries | Buffers 1.4 complete frames (margin for back-pressure) |
| Implementation | Distributed RAM | Small depth; lower latency than BRAM for $\leq 64$ entries |
| Flags | `full`, `empty`, `count[5:0]` | Parser reads when `empty` = '0'; overflow detection |

**Design Choice: Distributed RAM vs. Block RAM**

For FIFOs of 64 entries or fewer, Vivado will infer distributed RAM (implemented in LUT resources) rather than consuming a block RAM tile. This is the correct choice here: the Artix-7 100T has only 135 block RAM tiles (4,860 Kb total), and each one is a precious resource that should be reserved for larger storage needs in the feature extraction datapath (Week 3). A 64×8 distributed RAM FIFO consumes only 8 LUTs—negligible against the 63,400 available.

### 3.5 Week 1 Integration

By the end of Week 1, the following modules should be implemented and individually verified in simulation:

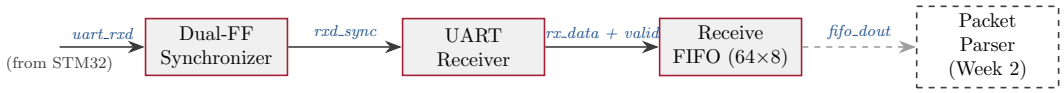


Figure 3: Week 1 pipeline: synchronizer → UART receiver → receive FIFO. The dashed parser block is implemented in Week 2.

**Exercise: UART Receiver Simulation**

Create a VHDL testbench that:

1. Generates a stimulus signal mimicking the STM32 transmitting the sequence `0x7E, 0x29, 0xA1` (the start delimiter, a frame length, and part of a timestamp) at 115200 baud. Include proper start and stop bits.
2. Verifies that the UART receiver outputs the correct bytes with the correct `rx_valid` timing.
3. Introduces a ±2% baud rate deviation in the stimulus and confirms the receiver still decodes correctly (testing the oversampling tolerance).
4. Injects a framing error (missing stop bit) and verifies that `rx_error` is asserted.

Record waveform screenshots showing correct byte capture, FIFO write, and error detection.

**Week 1 Deliverable**

**UART Receiver Design and Simulation Report** — A document containing:

1. VHDL source code for the synchronizer, UART receiver, and receive FIFO modules.
2. Entity port descriptions and design parameter justifications.
3. Simulation waveform captures showing correct byte reception, FIFO operation, and error detection.
4. Baud rate tolerance analysis: demonstrate correct reception at $\pm 2\%$ baud deviation and identify the failure threshold.

# 4 Week 2: Packet Parser and Frame Integrity Verification

## 4.1 Objectives

- Implement a state-machine-based packet parser that detects the `0x7E` frame start delimiter and extracts the 43-byte receiver frame structure defined in Module 1.
- Implement hardware CRC-8 verification for the receiver frame (transport integrity) and CRC-16 verification for the embedded beacon payload (source integrity).
- Integrate the parser with the Week 1 UART receiver and FIFO.
- Simulate the integrated pipeline with valid and deliberately corrupted frames.

## 4.2 Frame Structure Recap

The parser must handle two nested data structures. The outer structure is the receiver-to-FPGA forwarding frame defined in Module 1 (Table 6):

Table 5: Receiver-to-FPGA forwarding frame (43 bytes, from Module 1).

| Field | Offset | Size | Description |
|---|---|---|---|
| Start Delimiter | 0 | 1 B | Fixed `0x7E` |
| Frame Length | 1 | 1 B | Remaining frame length (excluding start delimiter and CRC) |
| RX Timestamp | 2 | 4 B | Receiver system tick (ms) |
| RSSI | 6 | 1 B | Received signal strength (dBm, two's complement) |
| SNR | 7 | 1 B | Signal-to-noise ratio (dB, signed) |
| Freq. Error | 8 | 2 B | Frequency error (Hz, signed 16-bit) |
| Beacon Payload | 10 | 32 B | Complete beacon packet (see inner structure) |
| Frame CRC-8 | 42 | 1 B | CRC-8 over bytes 1–41 |

The inner structure is the beacon packet embedded at offset 10 (Module 1, Table 4):

## 4.3 Packet Parser State Machine

The parser reads bytes sequentially from the receive FIFO and walks through the frame structure field by field. A byte counter tracks the current position within the frame, and dedicated registers capture each field as it arrives.

### 4.3.1 Parser Entity Interface

## 4.4 Hardware CRC Implementation

The pipeline performs two levels of CRC verification. The CRC-8 check validates transport integrity (did the bytes survive the UART link without corruption?), while the CRC-16 check validates source integrity (did the beacon construct the packet correctly?). Both checks execute in hardware as the bytes stream through the parser, adding zero latency to the overall frame processing time.

Table 6: Beacon packet structure (32 bytes, embedded within forwarding frame).

| Field | Offset | Size | Description |
|---|---|---|---|
| Preamble Sync | 0 | 2 B | `0xAE5A` (frame detection) |
| Beacon ID | 2 | 1 B | Unique beacon identifier |
| Sequence Number | 3 | 2 B | Packet loss detection counter |
| TX Power | 5 | 1 B | Transmit power (dBm) |
| Modulation Code | 6 | 1 B | 0x01=LoRa, 0x02=FSK, 0x03=OOK |
| Spreading Factor | 7 | 1 B | SF7–SF12; 0x00 if non-LoRa |
| Bandwidth Code | 8 | 1 B | 0x00=125kHz, 0x01=250kHz |
| Timestamp | 9 | 4 B | Beacon-side system tick (ms) |
| Payload | 13 | 17 B | Pseudo-random or user data |
| CRC-16 | 30 | 2 B | CRC over bytes 0–29 |

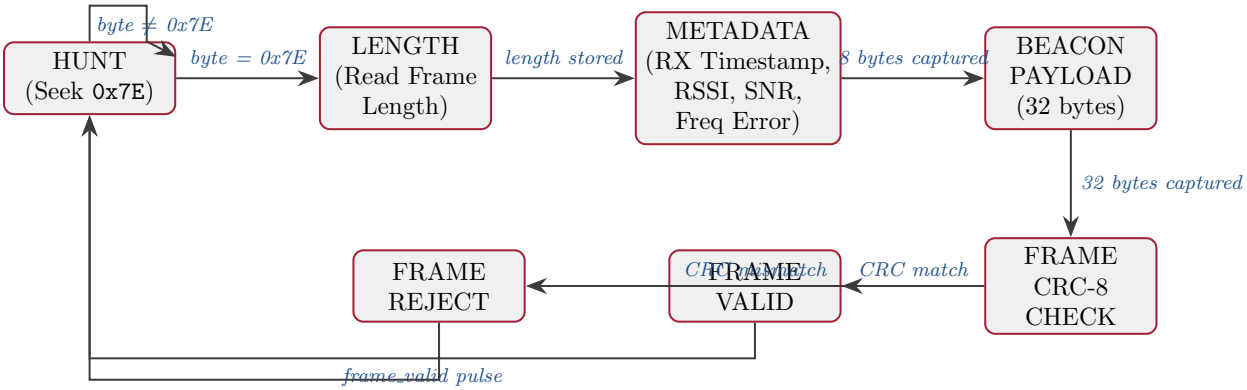


Figure 4: Packet parser state machine. The HUNT state discards bytes until the start delimiter is found. The parser then progresses through fixed-length fields sequentially, culminating in CRC verification. On CRC failure, the parser returns to HUNT without asserting `frame_valid`.

### 4.4.1 CRC-8 (Frame Transport Integrity)

The receiver frame uses CRC-8 with the polynomial:

$$G(x) = x^8 + x^2 + x + 1 \quad \text{(0x07, CRC-8/ITU)} \tag{5}$$

The CRC is computed over bytes 1–41 of the frame (everything between the start delimiter and the CRC byte itself). The hardware implementation uses a byte-wide combinational CRC step function: each time a new byte arrives from the parser, the current CRC register is updated in a single clock cycle.

The byte-wide CRC-8 update can be expressed as a set of XOR equations derived from the polynomial. For an 8-bit input byte $D[7{:}0]$ and current CRC state $C[7{:}0]$, the next CRC state $C'[7{:}0]$ is a purely combinational function:

$$C'[i] = f_i(C[7{:}0], D[7{:}0]) \quad \text{for } i = 0, 1, \ldots, 7 \tag{6}$$

Table 7: Packet parser entity ports.

| Port | Dir | Width | Description |
|---|---|---|---|
| `clk` | in | 1 | 100 MHz system clock |
| `rst` | in | 1 | Synchronous active-high reset |
| `fifo_data` | in | 8 | Byte from receive FIFO |
| `fifo_empty` | in | 1 | FIFO empty flag |
| `fifo_rd` | out | 1 | FIFO read enable (parser requests next byte) |
| `frame_valid` | out | 1 | One-cycle pulse: complete frame passed CRC |
| `frame_reject` | out | 1 | One-cycle pulse: frame failed CRC |
| `rx_timestamp` | out | 32 | Receiver timestamp field |
| `rssi` | out | 8 | RSSI field |
| `snr` | out | 8 | SNR field |
| `freq_error` | out | 16 | Frequency error field |
| `beacon_id` | out | 8 | Beacon ID from embedded payload |
| `seq_num` | out | 16 | Beacon sequence number |
| `tx_power` | out | 8 | Beacon TX power |
| `mod_code` | out | 8 | Modulation code |
| `spread_factor` | out | 8 | Spreading factor |
| `bw_code` | out | 8 | Bandwidth code |
| `beacon_timestamp` | out | 32 | Beacon-side timestamp |
| `beacon_crc_ok` | out | 1 | Inner CRC-16 verification result |

where each $f_i$ is an XOR tree that can be generated automatically by tools or derived from the polynomial long division.

**Implementation Strategy: CRC XOR Equations**

Rather than implementing CRC bit-by-bit (which would require 8 clock cycles per byte), use a pre-computed byte-wide XOR network. The open-source tool `pycrc` or the online Output-Logic CRC Generator can produce the VHDL combinational equations for any polynomial and data width. For CRC-8/0x07 with 8-bit data input, this yields 8 XOR equations, each involving a subset of the 16 input bits ($C$[7:0] and $D$[7:0]). The entire update computes in a single combinational step.

### 4.4.2 CRC-16 (Beacon Payload Integrity)

The beacon packet uses CRC-16-CCITT:

$$G(x) = x^{16} + x^{12} + x^5 + 1 \quad \text{(0x1021)} \tag{7}$$

This CRC is computed over bytes 0–29 of the beacon payload (the first 30 bytes within the

32-byte beacon packet embedded at frame offset 10). The same byte-wide combinational approach applies: a 16-bit CRC register is updated once per byte as the parser streams through the beacon payload region.

### 4.4.3 CRC Verification Timing

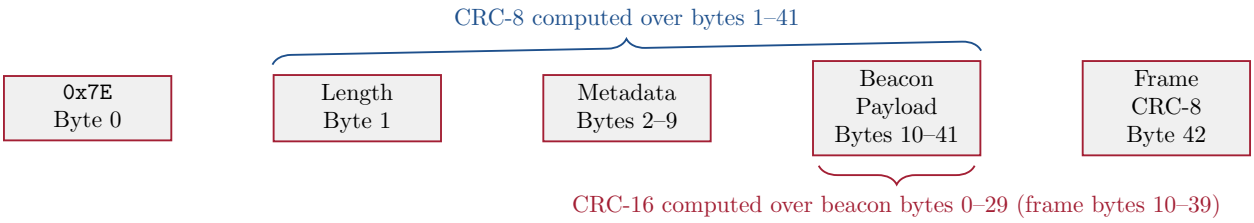


Figure 5: CRC coverage within the forwarding frame. CRC-8 covers the entire frame body. CRC-16 covers the beacon payload independently. Both are computed concurrently as bytes stream through the parser.

**Common Pitfalls**

- **CRC initialization:** CRC-8/ITU initializes to `0x00`; CRC-16-CCITT initializes to `0xFFFF`. Incorrect initialization will produce a valid-looking but wrong CRC for every frame, causing 100% rejection.
- **Byte ordering:** The CRC-16 check bytes in the beacon packet are transmitted MSB-first. Ensure the comparison logic accounts for the byte order of the received CRC field versus the computed residual.
- **Running CRC during non-covered bytes:** The CRC-8 engine must *not* include byte 0 (the start delimiter) in its computation. The CRC-16 engine must only process the 30 beacon data bytes (frame bytes 10–39), not the 2-byte CRC-16 field itself (bytes 40–41). Use the parser's byte counter to gate the CRC update enable signals.

### 4.5 Integrated Parser Simulation

**Exercise: Parser Verification with Module 1 Data**

Create a simulation testbench that:

1. Feeds a sequence of valid 43-byte frames (recorded or hand-constructed from Module 1 packet format) through the UART receiver and into the parser. Verify that all parsed fields match expected values and `frame_valid` is asserted for each frame.
2. Inserts a single corrupted byte (bit flip) in the metadata region of one frame. Verify that CRC-8 fails and `frame_reject` is asserted while subsequent valid frames still parse correctly.
3. Inserts a corrupted byte within the beacon payload region. Verify that the frame-level CRC-8 still fails (the corruption propagates through both CRC engines) and the beacon-level CRC-16 also reports failure via `beacon_crc_ok` = '0'.
4. Sends a stream of bytes with *no* `0x7E` delimiter for 100 bytes, then sends a valid frame. Verify that the parser remains in HUNT state during the garbage bytes and correctly synchronizes when the valid frame arrives.
5. Tests back-to-back frames with zero inter-frame gap. Verify the parser immediately re-enters HUNT and captures the second frame.

**Week 2 Deliverable**

**Packet Parser and CRC Verification Report** — A document containing:

1. VHDL source code for the packet parser state machine, CRC-8 module, and CRC-16 module.
2. State machine diagram with transition conditions and output assignments.
3. Simulation waveform captures demonstrating: valid frame parsing, CRC-8 rejection, CRC-16 rejection, synchronization recovery, and back-to-back frame handling.
4. A table comparing expected vs. actual parsed field values for at least 5 distinct test frames.

# 5 Week 3: Feature Extraction Datapath

## 5.1 Objectives

- Define a fixed-format feature vector that captures the signal characteristics needed by the ML classification pipeline in Module 4.
- Implement a feature extraction module that populates the feature vector from parsed packet fields.
- Design a dual-port block RAM storage structure for accumulating feature vectors from multiple beacons.
- Implement beacon prioritization logic based on received signal quality metrics.
- Provide a host-readable output interface for exporting feature vectors to the ML pipeline.

## 5.2 Feature Vector Design

The feature vector is the data product of the FPGA pipeline—the structured representation of each received beacon transmission that will serve as input to the machine learning classifiers in Module 4. Its design is a cross-domain decision: the FPGA designer must include enough information for the ML engineer to build accurate classifiers, but the vector must remain compact enough to store and export efficiently within the FPGA's resource budget.

**Design Principle: Features as Cross-Domain Interface**

The feature vector is the primary data contract between the FPGA pipeline (Module 2) and the ML classification pipeline (Module 4). Decisions made here—which fields to include, their precision, and their ordering—directly constrain the preprocessing and model architecture decisions students make two modules later. This is a concrete example of the cross-domain interface reasoning the framework is designed to teach: an FPGA design choice shapes an ML workflow.

### 5.2.1 Feature Vector Format

### 5.2.2 Computed Features

Three fields in the feature vector are not directly extracted from the packet but are computed by the FPGA logic from the current and previous observations:

- **Inter-Arrival Time:** The difference between the current RX timestamp and the RX timestamp of the last valid packet from the same Beacon ID. This feature captures transmission scheduling regularity, which may differ between beacon modes (TDMA vs. ALOHA) and is a useful discriminator for the ML pipeline.

$$\Delta t_{\text{IAT}}^{(k)} = T_{\text{rx,current}}^{(k)} - T_{\text{rx,previous}}^{(k)} \tag{8}$$

  where $k$ is the Beacon ID.

Table 8: Feature vector format (32 bytes per beacon observation).

| **Field** | **Offset** | **Size** | **Description / ML Relevance** |
|---|---|---|---|
| Beacon ID | 0 | 1 B | Class label for emitter identification tasks. |
| Modulation Code | 1 | 1 B | Ground-truth label for modulation classification. |
| Spreading Factor | 2 | 1 B | Ground-truth label for SF detection tasks. |
| Bandwidth Code | 3 | 1 B | RF configuration parameter. |
| TX Power | 4 | 1 B | Transmitted power (enables path loss estimation). |
| RSSI | 5 | 1 B | Primary signal strength feature (signed). |
| SNR | 6 | 1 B | Signal quality feature (signed). |
| Freq. Error | 7 | 2 B | Carrier offset feature (signed, Hz). |
| RX Timestamp | 9 | 4 B | Absolute reception time. |
| Beacon Timestamp | 13 | 4 B | Transmission time (enables latency calculation). |
| Sequence Number | 17 | 2 B | Packet loss and ordering feature. |
| Payload CRC Status | 19 | 1 B | 0x01 = pass, 0x00 = fail (data quality indicator). |
| Inter-Arrival Time | 20 | 4 B | Time since last packet from this beacon ID (computed). |
| RSSI Delta | 24 | 2 B | Change in RSSI from previous observation (computed, signed). |
| Observation Count | 26 | 2 B | Running count of observations from this beacon ID. |
| Reserved | 28 | 4 B | Padding for 32-byte alignment; available for future features. |

- **RSSI Delta:** The signed difference between the current RSSI and the RSSI of the previous observation from the same beacon. Large RSSI fluctuations may indicate multipath fading, obstruction, or power control changes—all of which are relevant to anomaly detection tasks.

$$\Delta\text{RSSI}^{(k)} = \text{RSSI}^{(k)}_{\text{current}} - \text{RSSI}^{(k)}_{\text{previous}} \tag{9}$$

- **Observation Count:** A running counter of valid frames received from each Beacon ID since reset. This provides a measure of link reliability and can serve as a weight or confidence indicator in the ML pipeline.

**Common Pitfalls**

- **First-observation edge case:** When a beacon is seen for the first time (observation count = 1), there is no previous timestamp or RSSI. Set Inter-Arrival Time to `0xFFFFFFFF` (sentinel) and RSSI Delta to `0x0000`. The ML preprocessing code in Module 4 must be aware of these sentinel values.
- **Timestamp rollover:** The STM32 system tick is a 32-bit millisecond counter that rolls over after ∼49.7 days. For benchtop sessions this is not a concern, but the subtraction logic should use unsigned arithmetic to handle rollover correctly in principle.
- **Signed vs. unsigned:** RSSI (two's complement) and Freq. Error (signed 16-bit) are signed quantities. Ensure the VHDL arithmetic uses `signed` types from `ieee.numeric_std` for delta computations.

## 5.3 Per-Beacon State Registers

To compute the derived features, the FPGA must maintain per-beacon state. A small register file indexed by Beacon ID stores the previous observation's timestamp, RSSI, and the running observation count.

Table 9: Per-beacon state storage (7 bytes per beacon).

| Field | Size | Description |
|---|---|---|
| Previous RX Timestamp | 4 B | Last valid reception time for this Beacon ID |
| Previous RSSI | 1 B | Last RSSI value for this Beacon ID |
| Observation Count | 2 B | Running count of valid observations |

With Beacon IDs ranging from `0x01` to `0xFF` (255 possible beacons), the state register file requires $255 \times 7 = 1{,}785$ bytes. This fits comfortably in a single 2 KB block RAM tile (one 18 Kb BRAM configured as $256 \times 8$, or a pair of them for wider access).

**Design Choice: Register File vs. Lookup Table**

For the expected benchtop scenario of 2–8 beacons, a simple array of registers would suffice and avoid the BRAM overhead. However, implementing the state as an addressable RAM indexed by Beacon ID scales cleanly to the full 255-beacon address space without modification. This is a deliberate pedagogical choice: students learn to design for the general case using parameterizable, memory-mapped structures rather than hard-coding for a specific beacon count. The BRAM cost (1 tile out of 135) is minimal.

## 5.4 Feature Storage: Dual-Port Block RAM

Completed feature vectors are written to a dual-port block RAM that serves as the handoff buffer between the FPGA pipeline and the host-side ML data collection. Port A is the FPGA write port (driven by the feature extraction logic); Port B is the read port (driven by the host interface for data export).

Table 10: Feature storage BRAM configuration.

| Parameter | Value | Rationale |
|---|---|---|
| Width | 256 bits (32 bytes) | One feature vector per address |
| Depth | 256 entries | One slot per possible Beacon ID |
| Total Size | 8 KB | Fits in 4× 18 Kb BRAM tiles (or 2× 36 Kb) |
| Port A | Write (FPGA pipeline) | Synchronous write on `frame_valid` |
| Port B | Read (host interface) | Asynchronous read for data export via UART |

#### 5.4.1 Write Logic

When the parser asserts `frame_valid`, the feature extraction module:

1. Reads the per-beacon state for the incoming Beacon ID from the state RAM.
2. Computes the three derived features (Inter-Arrival Time, RSSI Delta, Observation Count).
3. Assembles the 32-byte feature vector.
4. Writes the feature vector to the feature BRAM at address = Beacon ID.
5. Updates the per-beacon state RAM with the current timestamp, RSSI, and incremented count.

This sequence completes in a small fixed number of clock cycles (state read latency + computation + write = ∼4–6 cycles at 100 MHz), which is negligible compared to the ∼3.73 ms frame arrival interval at 115200 baud.

#### 5.4.2 Read / Export Interface

For Module 2, the export interface is a simple UART transmitter that sends feature vectors to the host PC on demand (triggered by a pushbutton or a host-side UART command). The host-side Python script stores the received feature vectors as CSV or binary records for use in Module 4. The full export protocol is finalized during Module 3 integration, but the BRAM read port and a minimal UART TX module should be implemented now to enable early testing.

### 5.5 Beacon Prioritization Logic

As a stretch goal for advanced students, implement a simple prioritization scheme that ranks active beacons by signal quality. The priority score for each beacon can be computed as:

$$P^{(k)} = \alpha \cdot \mathrm{RSSI}^{(k)} + \beta \cdot \mathrm{SNR}^{(k)} - \gamma \cdot |\Delta\mathrm{RSSI}^{(k)}| \tag{10}$$

where $\alpha$, $\beta$, and $\gamma$ are configurable 8-bit weight registers. The beacon with the highest priority score is indicated on the Nexys A7 seven-segment display, providing a real-time visual indicator of the "best" signal source. This exercise introduces students to parameterized hardware arithmetic and prepares them for the concept of hardware-accelerated inference in Module 4.

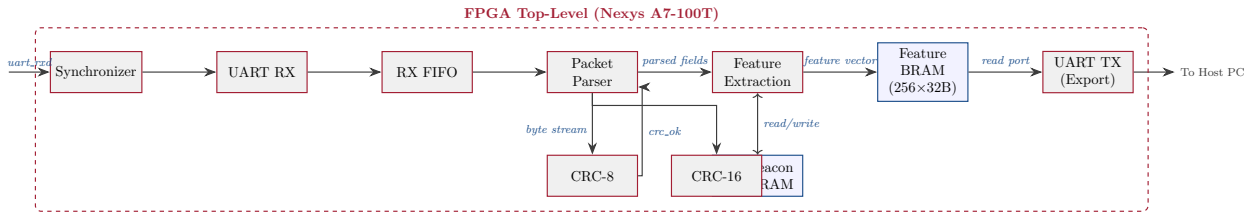


Figure 6: Complete Module 2 FPGA pipeline architecture. Solid blocks are implemented across Weeks 1–3. The UART TX export module enables early data extraction for ML pipeline prototyping.

## 5.6 Week 3 Pipeline Architecture

**Exercise: Feature Vector Verification**

Using two beacons transmitting in fixed-interval mode (as configured in Module 1 Week 3):

1. Run the full FPGA pipeline in simulation for 30 simulated seconds of beacon traffic (approximately 15 packets per beacon at 2-second intervals).
2. Export the feature BRAM contents and verify:
   - Beacon IDs `0x01` and `0x02` each have entries in the BRAM.
   - RSSI values differ between beacons (reflecting the TX power difference from Module 1).
   - Observation counts increment correctly for each beacon.
   - Inter-arrival times are approximately 2000 ms (± UART/processing jitter).
   - RSSI Delta values are near zero for fixed-position, fixed-power beacons.
3. Introduce a 5 dB TX power reduction on Beacon B midway through the simulation. Verify that the RSSI Delta becomes negative and the feature vector captures the transition.

**Week 3 Deliverable**

**Feature Extraction Datapath Design Document** — A document containing:

1. Feature vector format specification with field descriptions, sizes, and ML relevance.
2. VHDL source code for the feature extraction module, per-beacon state RAM, and feature BRAM.
3. Block diagram of the complete pipeline from UART input to feature BRAM output.
4. Simulation results demonstrating correct feature computation for multi-beacon scenarios, including computed features (inter-arrival time, RSSI delta, observation count).
5. Resource estimate: anticipated LUT, FF, and BRAM usage for the current design.

# 6 Week 4: Simulation, Verification, and Resource Optimization

## 6.1 Objectives

- Build comprehensive end-to-end testbenches that exercise the full pipeline from raw UART stimulus to feature vector output.
- Verify the pipeline against recorded Module 1 beacon data to confirm compatibility with the physical system.
- Analyze Vivado implementation reports to understand resource utilization, timing margins, and critical paths.
- Optimize the design to meet timing closure and minimize resource usage where appropriate.
- Document the complete RTL design for handoff to Module 3 system integration.

## 6.2 End-to-End Testbench Architecture

The comprehensive testbench wraps the entire FPGA top-level module and provides stimulus at the UART serial input while monitoring outputs at the feature BRAM export interface.

### 6.2.1 Stimulus Generation

The testbench reads stimulus data from a text file containing the byte sequences recorded during Module 1 validation. This approach ensures that the RTL is verified against the exact data the physical system produces, not idealized test vectors.

**Stimulus File Format**

Use a simple text file with one hexadecimal byte per line. The testbench reads each byte, serializes it according to the UART protocol (start bit, 8 data bits LSB-first, stop bit), and drives the `uart_rxd` signal with proper 115200-baud timing. This file-driven approach allows the same testbench to be reused with different data captures without modifying VHDL code. Example `stimulus.txt`:

```
7E 29 00 00 0F A0 C4 08 FF FE ...
```

### 6.2.2 Output Checking

The testbench includes an automated checker that compares the parsed field outputs and feature vector contents against a reference file of expected values. Any mismatch triggers an assertion failure with the byte offset and expected/actual values, enabling rapid debugging.

## 6.3 Verification Scenarios

The following scenarios must be covered by the testbench suite:

## 6.4 Resource Utilization Analysis

After synthesis and implementation in Vivado, students analyze the utilization report to understand how their design fits within the Artix-7 resource budget.

Table 11: Required verification scenarios.

| # | Scenario | Verification Criteria |
|---|---|---|
| 1 | Normal operation (2 beacons, 20 frames each) | All fields parsed correctly; all CRCs pass; feature vectors stored and exported. |
| 2 | CRC-8 failure (1 corrupted frame in 20) | Corrupted frame rejected; all other frames accepted; pipeline resynchronizes. |
| 3 | CRC-16 failure (beacon payload corruption) | Frame-level CRC-8 also fails; `beacon_crc_ok` = '0'; pipeline recovers. |
| 4 | Garbage bytes before valid frame | Parser remains in HUNT; synchronizes on first valid `0x7E`; no false positives. |
| 5 | Back-to-back frames (zero gap) | Both frames parsed correctly; no missed frames. |
| 6 | Maximum burst (5 frames, minimal spacing) | FIFO does not overflow; all frames parsed and stored. |
| 7 | Single beacon (long duration, 100 frames) | Observation count reaches 100; inter-arrival time stabilizes; no counter overflow. |
| 8 | Beacon ID collision (new beacon replaces old) | Feature BRAM overwrites previous entry; per-beacon state updates correctly. |

### 6.4.1 Expected Resource Budget

**Why Resource Awareness Matters at <1%**

The Module 2 pipeline consumes a tiny fraction of the Artix-7's capacity, which is by design. The deliberately low utilization at this stage teaches two lessons. First, it demonstrates that a well-structured ingestion pipeline is not the resource bottleneck—it is the downstream processing (filtering, inference, multi-channel reception in later modules) that will challenge the fabric. Second, it establishes a baseline against which students can measure the resource cost of design decisions in Module 3 (e.g., adding a second UART receiver for a second receiver station) and Module 4 (e.g., implementing an on-FPGA neural network inference engine). Understanding the resource budget *before* it becomes tight is essential to making good trade-off decisions when it does.

## 6.5 Timing Analysis

Students must review the Vivado timing summary report after implementation and understand the following metrics:

- **Worst Negative Slack (WNS):** Must be $\geq 0$ ns for timing closure. At 100 MHz (10 ns period), the combinational path between any two registers must complete within the setup time margin. For this design, WNS should be comfortably positive ($> 3$ ns slack expected).
- **Worst Hold Slack (WHS):** Must be $\geq 0$ ns. Hold violations indicate that data arrives

Table 12: Estimated resource utilization for the Module 2 pipeline.

| **Module** | **LUTs** | **FFs** | **BRAM (18Kb)** | **Notes** |
|---|---|---|---|---|
| Synchronizer | 2 | 2 | 0 | Two flip-flops |
| UART Receiver | ∼80 | ∼50 | 0 | State machine + counters |
| Receive FIFO (64×8) | ∼20 | ∼15 | 0 | Distributed RAM |
| Packet Parser | ∼200 | ∼300 | 0 | State machine + field registers |
| CRC-8 Engine | ∼30 | ∼10 | 0 | Combinational XOR network |
| CRC-16 Engine | ∼50 | ∼20 | 0 | Combinational XOR network |
| Feature Extraction | ∼150 | ∼200 | 0 | Arithmetic + muxing |
| Per-Beacon State RAM | 0 | 0 | 1 | 256×56 bits |
| Feature BRAM | 0 | 0 | 4 | 256×256 bits |
| UART TX (Export) | ∼60 | ∼40 | 0 | Byte serializer |
| **Total Estimate** | **∼600** | **∼640** | **5** | |
| **Artix-7 100T Available** | 63,400 | 126,800 | 270 | |
| **Utilization** | <1% | <1% | ∼2% | |

too quickly at the destination register. Vivado automatically inserts hold-fix buffers during place-and-route if needed.

- **Critical Path:** Identify which module contains the longest combinational path. The CRC XOR networks are likely candidates. If the critical path is in the CRC-16 engine, students should consider pipelining the CRC update (split the XOR tree across two register stages) as an optimization exercise.

**Exercise: Timing Closure Analysis**

1. Run Vivado synthesis and implementation for the complete Module 2 design targeting the XC7A100T-1CSG324C.
2. Record the WNS, WHS, and total negative slack (TNS) values.
3. Identify the critical path endpoints (source register → destination register) and the module it passes through.
4. If the critical path passes through the CRC-16 engine, insert a pipeline register to split the combinational logic and re-run implementation. Document the improvement in WNS.
5. Compare the post-implementation resource utilization against the estimates in Table 12. Explain any significant deviations.

### 6.6 Design Documentation Requirements

The Week 4 deliverable includes comprehensive documentation of the complete RTL design, prepared in a format that supports the Module 3 integration effort.

**Week 4 Deliverable**

**Simulation Verification and Resource Optimization Report** — A document containing:

1. End-to-end testbench VHDL source code with file-driven stimulus and automated output checking.
2. Simulation results for all 8 verification scenarios in Table 11, with waveform captures and pass/fail summary.
3. Vivado resource utilization report (post-implementation) with comparison to estimates.
4. Timing summary (WNS, WHS, critical path identification) and any optimization steps taken.
5. Complete RTL module hierarchy diagram with port-level interface descriptions.
6. Pin assignment and constraint file (`.xdc`) for the Nexys A7-100T, covering UART RX/TX pins, debug LEDs, and seven-segment display outputs.

# 7 Hardware and Software Reference

Table 13: Required hardware and software for Module 2.

| Item | Qty / Version | Purpose |
|---|---|---|
| Digilent Nexys A7-100T | 1 | Target FPGA board (Artix-7 XC7A100T) |
| USB Micro-B Cable | 1 | FPGA programming and UART communication |
| STM32 Nucleo-L476RG + RF Module | $\geq 1$ | Beacon/receiver for live data capture (optional in sim-only weeks) |
| AMD Vivado Design Suite | 2024.1+ (WebPACK) | Synthesis, implementation, simulation |
| Vivado Simulator (built-in) | — | Behavioral and post-synthesis simulation |
| ModelSim (optional) | — | Alternative HDL simulator |
| Python 3.10+ | — | Stimulus file generation and output verification scripts |
| Terminal Emulator | — | UART debug (PuTTY, Tera Term, or minicom) |

## 7.1 FPGA Pin Assignments

The following pin assignments extend the Module 1 loopback validation with the signals needed for the Module 2 pipeline:

# 8 Assessment Rubric

Table 14: Key FPGA pin assignments on the Nexys A7-100T.

| Signal | FPGA Pin | Connector | Description |
|---|---|---|---|
| `uart_rxd` (from STM32) | JA[1] | PMOD JA | Receiver MCU TX → FPGA RX |
| `uart_txd` (to host) | C4 | USB-UART | Feature export to host PC |
| `clk_100mhz` | E3 | On-board | 100 MHz system clock |
| `rst` | N17 | BTNC | Center pushbutton (active-high reset) |
| `export_trigger` | M18 | BTNU | Upper pushbutton (trigger feature export) |
| `led[15:0]` | H17–V14 | On-board LEDs | Debug: frame count, FIFO status, CRC status |
| `seg[6:0]` + `an[7:0]` | Various | 7-seg display | Beacon ID and priority display |

Table 15: Module 2 assessment rubric.

| Assessment Item | Weight | Due |
|---|---|---|
| UART Receiver Design and Simulation Report (synchronizer, receiver, FIFO, baud tolerance) | 20% | Week 1 |
| Packet Parser and CRC Verification Report (state machine, CRC-8/16, integration simulation) | 25% | Week 2 |
| Feature Extraction Datapath Design Document (feature vector, state RAM, BRAM, multi-beacon) | 25% | Week 3 |
| Simulation Verification and Resource Optimization Report (testbench suite, utilization, timing) | 30% | Week 4 |

# 9 Recommended Resources

## 9.1 Datasheets and Technical References


1. Xilinx/AMD, "Artix-7 FPGAs Data Sheet: DC and AC Switching Characteristics," DS181, 2023.
2. Digilent, "Nexys A7 Reference Manual," Rev. C, 2022.
3. Xilinx/AMD, "7 Series FPGAs Memory Resources User Guide," UG473, 2023. (Block RAM configuration and dual-port usage.)
4. Xilinx/AMD, "7 Series FPGAs Configurable Logic Block User Guide," UG474, 2023. (Distributed RAM, LUT structure, and carry chains.)
5. Xilinx/AMD, "Vivado Design Suite User Guide: Synthesis," UG901, 2023. (Synthesis attributes including `ASYNC_REG`.)

### 9.2 VHDL Design References

1. P. P. Chu, *FPGA Prototyping by VHDL Examples: Xilinx Artix-7 Edition*, 2nd ed., Wiley, 2017. (Chapters on UART design, FIFO structures, and testbench methodology.)

2. R. E. Haskell and D. M. Hanna, *Digital Design Using Digilent FPGA Boards: VHDL / Vivado Edition*, LBE Books, 2019. (Nexys A7-specific examples and constraint file templates.)

3. P. J. Ashenden, *The Designer's Guide to VHDL*, 3rd ed., Morgan Kaufmann, 2008. (Comprehensive VHDL language reference.)

### 9.3 CRC Implementation References

1. W. W. Peterson and D. T. Brown, "Cyclic Codes for Error Detection," *Proceedings of the IRE*, vol. 49, no. 1, pp. 228–235, Jan. 1961.

2. T. Jejekel, "pycrc — Free, Easy to Use CRC Calculator and C Source Code Generator." [Online]. Available: https://pycrc.org/

3. OutputLogic, "CRC Generator." [Online]. Available: https://outputlogic.com/?page_id=321 (Generates VHDL/Verilog CRC modules for arbitrary polynomials and data widths.)

### 9.4 Application Notes

1. Xilinx/AMD, "XAPP585: UART Lite Core," Application Note, 2023. (Reference UART implementation for Xilinx FPGAs.)

2. Xilinx/AMD, "XAPP495: Implementing a Serial Communication Protocol Using the SPI Interface," Application Note, 2023.

# 10 Transition to Module 3

Upon successful completion of Module 2, students will have:

- A fully simulated and verified FPGA signal processing pipeline: UART ingestion → packet parsing → CRC verification → feature extraction → BRAM storage.
- Hardware CRC engines that validate both transport-level and source-level data integrity in real time.
- A feature vector format designed explicitly for the ML classification pipeline, with both directly-extracted and FPGA-computed features.
- Dual-port BRAM storage supporting concurrent write (from pipeline) and read (for host export), sized for up to 255 simultaneous beacons.
- Comprehensive simulation testbench coverage across 8 verification scenarios with file-driven stimulus.
- Characterized resource utilization and timing margins, establishing a baseline for Module 3 expansion.
- Pin assignments and constraint files ready for hardware deployment.

Module 3 will build directly on these foundations by deploying the RTL to the physical Nexys A7 and integrating it with the STM32 receiver subsystem from Module 1. The primary challenge shifts from design correctness (verified in simulation) to physical integration: the FPGA must now process real UART data from the STM32 at the actual baud rate, handle timing variations and noise that simulation cannot model, and deliver feature vectors reliably to the host machine. Synchronization mismatches, protocol errors, and throughput bottlenecks that were invisible in per-subsystem development are expected to surface during this integration phase—diagnosing and resolving them is the central learning objective of Module 3.

The feature BRAM and export interface implemented in Week 3 become the data source for the structured data collection campaigns in Module 3, which produce the labeled RF dataset that drives the entire ML pipeline in Module 4.

End of Module 2 Lesson Plan